STREAMIX

Streamix

Technical Architecture & Engineering Reference

TECHNICAL DOCUMENT

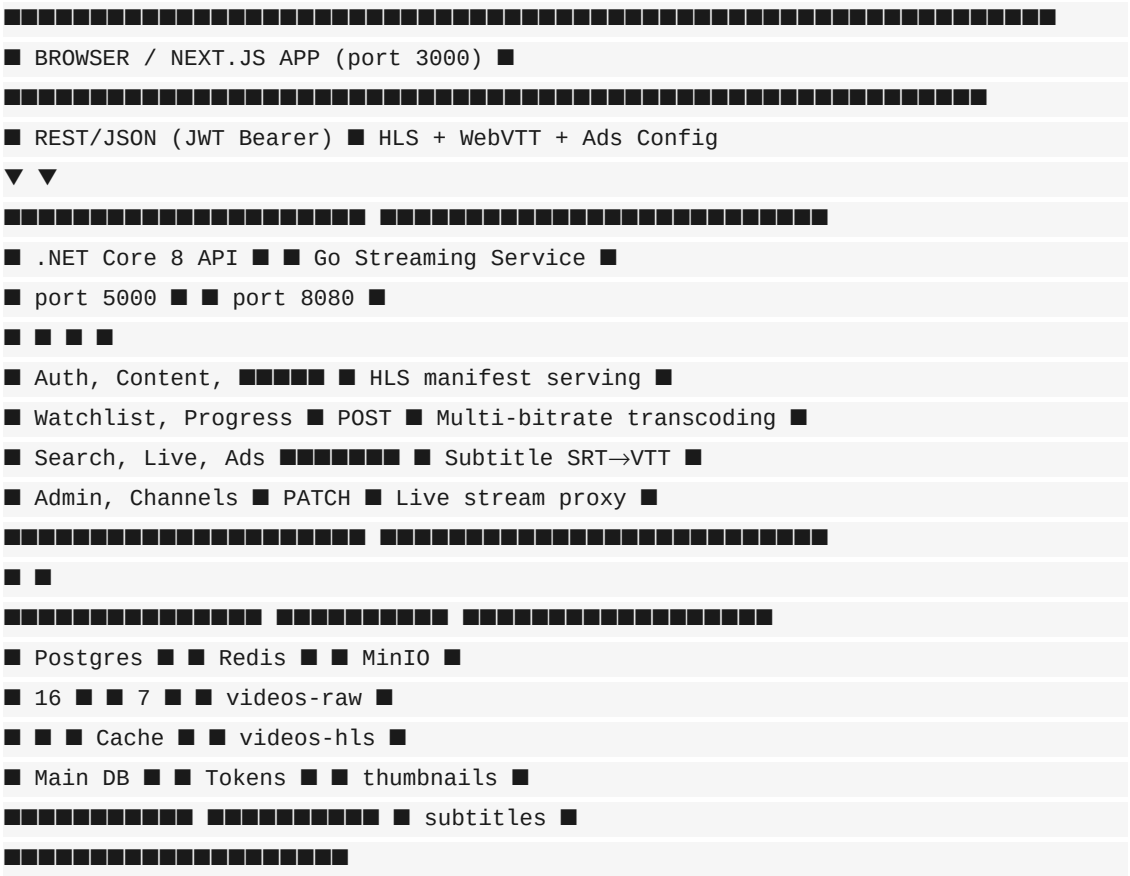
Version 2.0 · June 10, 2026

Table of Contents

1. System Architecture Overview

Streamix is a three-service microservices platform. Each service is built in the language best suited to its workload and communicates over HTTP. All services are containerised and share MinIO object storage and Redis for caching and session management.

Service Architecture Diagram



Service Responsibilities

.NET Core 8 API	Authentication (JWT), users, profiles, subscription management, content catalogue, watchlist, watch progress, search, live events, channels, admin operations, ad configuration endpoint, and internal callbacks from Go. Only service with direct PostgreSQL access.
Go Streaming Service	Receives raw video paths from backend, runs FFmpeg to produce multi-bitrate HLS, uploads segments to MinIO, serves token-gated HLS manifests and segments. Handles live stream proxy and SRT→VTT subtitle conversion. No DB access.

Next.js 15 Frontend	Server-side and client-side React app using App Router. Server Components for browse/search pages; Client Components for player, ads, watchlist, and interactive UI. Proxies all API and streaming requests via Next.js rewrites (no browser CORS).

2. Technology Stack

Layer	Technology	Version	Rationale
Frontend Framework	Next.js	15.x	App Router, Server Components, rewrites
UI Language	TypeScript	5.x	Type safety across API boundaries
Styling	Tailwind CSS	3.x	Utility-first; dark theme with brand tokens
HLS Player	HLS.js	1.5.x	Adaptive bitrate, quality selection, DRM-ready
Server State	TanStack Query	5.x	Caching, background refetch, infinite scroll, ad config
Client State	Zustand	4.x	Auth store with localStorage persistence
Auth (FE)	NextAuth.js	5.x	App Router compatible; CredentialsProvider
Animations	Framer Motion	11.x	Card hover, modal transitions
Backend Framework	ASP.NET Core	8.0	High-performance, mature ecosystem
Architecture	Clean Architecture	—	Domain / Application / Infrastructure / API
CQRS	MediatR	12.x	Decoupled command/query handlers + pipeline
Validation	FluentValidation	11.x	Declarative rules, MediatR pipeline behaviour
ORM	EF Core + Npgsql	8.x	Code-first migrations, LINQ, PostgreSQL driver
Streaming Language	Go	1.24	Goroutines for parallel transcoding; low memory
HTTP Router (Go)	Chi	5.x	Lightweight, idiomatic, middleware-first
Transcoder	FFmpeg	6.x	Industry-standard multi-bitrate HLS
Database	PostgreSQL	16	tsvector, pg_trgm, uuid-ossp, GIN indexes
Cache / Sessions	Redis	7	API cache, JWT refresh tokens, ad config TTL
Object Storage	MinIO	latest	S3-compatible; self-hosted; presigned URLs

Reverse Proxy	Nginx	1.25	SSL termination, routing, large upload support
Containerisation	Docker / Compose	v5	7-service dev orchestration; Kubernetes for prod

3. Repository Structure

Monorepo under **streming/** — three service directories plus shared infrastructure configs:

```
streming/
■■■ .env.example
■■■ docker-compose.yml
■■■ docker-compose.dev.yml
■■■ Makefile
■■■ apps/
■ ■■■ frontend/ # Next.js 15
■ ■ ■■■ src/app/ # App Router pages
■ ■ ■■■ src/components/
■ ■ ■ ■■■ browse/ # HeroSection, ContentRow, ContentCard
■ ■ ■ ■■■ player/ # VideoPlayer, PlayerControls, AdPlayer, AdBanner
■ ■ ■ ■■■ live/ # LivePlayer, LiveBadge, LiveEventCard, ChannelCard
■ ■ ■ ■■■ title/ # TitleDetail, EpisodeList, WatchlistButton
■ ■ ■ ■■■ search/ # SearchBar, SearchResults
■ ■ ■ ■■■ layout/ # Navbar, Footer
■ ■ ■ ■■■ ui/ # Button, Input, Badge, Skeleton, Modal
■ ■ ■■■ src/hooks/ # useAdConfig, useWatchlist, useProfile, useDebounce
■ ■ ■■■ src/lib/api/ # client, auth, content, ads, live, channels...
■ ■ ■■■ src/store/ # Zustand auth store
■ ■ ■■■ src/types/ # content, user, live, api
■ ■ ■■■ src/middleware.ts # Edge middleware: route protection
■ ■ ■■■ Dockerfile
■ ■ ■■■ package.json
■ ■■■ backend/ # .NET Core 8
■ ■ ■■■ src/StreamingPlatform.API/
■ ■ ■ ■■■ Controllers/ # Auth, Content, Ads, Live, Channels, Admin...
■ ■ ■ ■■■ Middleware/ # ExceptionHandlingMiddleware
■ ■ ■ ■■■ appsettings.json # Jwt, Minio, Redis, Ads, StreamingService
■ ■ ■■■ src/StreamingPlatform.Application/
■ ■ ■ ■■■ Common/DTOs/
■ ■ ■ ■■■ Common/Interfaces/
■ ■ ■ ■■■ Common/Models/
■ ■ ■■■ src/StreamingPlatform.Domain/
■ ■ ■ ■■■ Entities/ # 14 entities
■ ■ ■ ■■■ Enums/ # ContentType, VideoQuality, ProcessingStatus...
■ ■ ■■■ src/StreamingPlatform.Infrastructure/
■ ■ ■ ■■■ Persistence/ # ApplicationDbContext + 14 EF configs
■ ■ ■ ■■■ Services/ # TokenService, CacheService, MinioStorageService...
■ ■ ■■■ Dockerfile
■ ■■■ streaming/ # Go 1.24
```

```
■ ■■■ cmd/server/main.go
■ ■■■ internal/
■ ■ ■■■ config/ handler/ service/ storage/ cache/
■ ■ ■■■ transcoder/ # ffmpeg, pipeline, worker_pool, subtitle
■ ■ ■■■ middleware/ # auth.go (JWT validation)
■ ■■■ pkg/models/models.go
■ ■■■ Dockerfile
■■■ infra/
■■■ postgres/init.sql # uuid-ossp, pg_trgm, unaccent extensions
■■■ redis/redis.conf
■■■ minio/setup.sh # create 4 buckets
■■■ nginx/nginx.conf # reverse proxy rules
```

4. Backend Service — .NET Core 8

Clean Architecture Layers

Domain	14 entities (User, Profile, Subscription, Content, Season, Episode, VideoAsset, Subtitle, Genre, ContentGenre, WatchlistItem, WatchProgress, Channel, LiveEvent). 5 enums: ContentType, VideoQuality, ProcessingStatus, SubscriptionTier, LiveStreamStatus. No framework dependencies.
Application	5 interfaces: IApplicationDbContext, ITokenService, ICacheService, IStorageService, IStreamingService. 15 DTOs. PaginatedList and Result models. MediatR + FluentValidation registered in DependencyInjection.cs.
Infrastructure	ApplicationDbContext with 14 IEntityTypeConfiguration files. TokenService (HS256 JWT), CacheService (Redis), MinioStorageService (presigned URLs), StreamingServiceClient (HttpClient with X-Internal-Key).
API	11 controllers: Auth, Content, Search, Progress, Watchlist, Users, Channels, Live, Admin, Ads. ExceptionHandlingMiddleware (RFC 7807 ProblemDetails). Swagger/OpenAPI with Bearer scheme. Health checks. Auto-migrate on startup.

appsettings.json Configuration Sections

Section	Key Settings
Jwt	Issuer, Audience, AccessTokenMinutes (15), RefreshTokenDays (7)
Minio	Endpoint, AccessKey, SecretKey, UseSSL, bucket names (4 buckets)
Redis	CacheTtlMinutes (10)
StreamingService	BaseUrl — internal URL of the Go service
Ads	Enabled (bool), MidRollIntervalMinutes, PreRoll.VideoUrl, PreRoll.SkipAfterSeconds, MidRoll.ImageUrl, MidRoll.Headline, MidRoll.DurationSeconds, MidRoll.CloseAfterSeconds
ConnectionStrings	DefaultConnection (PostgreSQL), Redis

JWT Design

Algorithm	HS256 — 256-bit secret from environment variable Jwt:Secret
Access Token TTL	15 minutes; claims: userId, email, profileId, subscriptionTier, jti
Refresh Token TTL	7 days; opaque random string stored in Redis as refresh:{token} → userId
Token Rotation	Each /auth/refresh call issues a new refresh token and invalidates the previous
Tier Enforcement	subscriptionTier claim read by both backend and Go service; ad system reads it on frontend

5. Streaming Service — Go 1.24

Key Files

File	Responsibility
cmd/server/main.go	Wire config, router, worker pool; start HTTP server
internal/config/config.go	Viper env-based configuration
internal/server/server.go	Chi router, CORS, auth middleware registration
internal/handler/stream_handler.go	GET /stream — manifest rewriting + segment redirect
internal/handler/transcode_handler.go	POST /internal/transcode — enqueue job
internal/handler/live_handler.go	GET /live — low-latency HLS proxy for live channels
internal/handler/health_handler.go	GET /health — liveness probe
internal/service/stream_service.go	Resolve MinIO paths, check subscription tier
internal/service/transcode_service.go	FFmpeg pipeline orchestration
internal/storage/minio_client.go	GetObject, PutObject, PresignedGet, PresignedPut
internal/cache/redis_client.go	Token cache, content metadata cache
internal/transcoder/ffmpeg.go	FFmpeg multi-bitrate command builder
internal/transcoder/pipeline.go	Parallel quality-level runner (errgroup)
internal/transcoder/worker_pool.go	Bounded goroutine pool (default: 3 concurrent FFmpeg)
internal/transcoder/subtitle.go	Pure-Go SRT → WebVTT converter
internal/middleware/auth.go	JWT validation (shared HS256 secret with backend)
pkg/models/models.go	TranscodeJob, StreamToken, TranscodeResult structs

Multi-Bitrate FFmpeg Pipeline

A single FFmpeg invocation produces all three quality levels in parallel via **filter_complex**, outputting 6-second HLS segments:

```
// Three renditions in one pass:
// 360p → 800k video + 96k audio → v0/seg*.ts
// 720p → 2800k video + 128k audio → v1/seg*.ts
// 1080p → 5000k video + 192k audio → v2/seg*.ts

// Output in MinIO videos-hls bucket:
// {assetId}/master.m3u8 (variant playlist)
// {assetId}/v0/prog_index.m3u8 + seg000.ts ...
```

```
// {assetId}/v1/prog_index.m3u8 + seg000.ts ...
// {assetId}/v2/prog_index.m3u8 + seg000.ts ...
```

HLS Token-Gating Flow

```
1. Client → GET /stream/{contentId}/master.m3u8?token=JWT
2. Go → Validate JWT; check subscriptionTier ≥ content.requiredTier
3. Go → Fetch master.m3u8 from MinIO
4. Go → Rewrite each segment URL:
/stream/{contentId}/v{q}/{seg}.ts?token=JWT
5. Go → Return rewritten playlist (200 OK)

6. Client → GET /stream/{contentId}/v1/seg003.ts?token=JWT
7. Go → Validate JWT; generate presigned MinIO GET URL (1hr expiry)
8. Go → 302 redirect to presigned URL
(MinIO serves bytes; Go does NOT proxy the stream)
```

6. Frontend — Next.js 15

App Router Pages

Route	Rendering	Description
/	Server	Redirect to /browse
/(auth)/login	Client	CredentialsProvider login form (react-hook-form + zod)
/(auth)/register	Client	Registration form
/browse	Server	Hero + ContentRows by genre; parallel data fetching
/browse/[genre]	Server	Genre-filtered grid with GenreFilter pills
/title/[contentId]	Server	TitleDetail: backdrop, synopsis, episodes, watchlist button
/watch/[contentId]	Client	Full-screen VideoPlayer; ad system active for Free tier
/search	Client	Debounced search bar + TanStack Query result grid
/live	Server	Live Now / Upcoming / TV Channels sections
/live/[eventId]	Client	LivePlayer (low-latency HLS, no seek bar)
/sports	Server	Sport-type filter tabs (Football, Basketball, F1, etc.)
/tv	Server	TV channel category grid
/profile/select	Client	'Who's Watching?' avatar picker
/profile/[profileId]	Client	Profile settings: name, maturity rating, language, PIN
/profile/[profileId]/watchlist	Server	Watchlist grid for active profile
/account	Client	Subscription tier, billing history, password change

Key Hooks

Hook	File	Purpose
useAdConfig	src/hooks/useAdConfig.ts	Fetch GET /api/ads/config via React Query; enabled only for Free tier; staleTime 5min
useWatchlist	src/hooks/useWatchlist.ts	Optimistic add/remove via useMutation; refetch on settle
useProfile	src/hooks/useProfile.ts	Switch active profile; update Zustand store and JWT claim

useProgressSync	src/components/player/useProgressSync.ts	Debounced 10s progress PUT; fires on pause and unmount
useDebounce	src/hooks/useDebounce.ts	Debounce helper used by SearchBar

VideoPlayer Architecture

HLS Library	HLS.js v1.5 with native Safari fallback via video.canPlayType()
Auth on segments	xhrSetup callback injects Authorization Bearer header
Quality switching	hls.currentLevel = n (-1 = auto); level list from hls.levels on MANIFEST_PARSED
Subtitles	elements added post-attach; VTT files served from MinIO public bucket
Resume playback	videoRef.currentTime = startPosition after HLS attach
Keyboard shortcuts	Space/K=play-pause, ←/→=±10s, M=mute, F=fullscreen, ↑/↓=volume
Controls hide	Hidden after 3s inactivity via setTimeout; shown on mousemove
Ad integration	showPreRoll state triggers AdPlayer before first play; showMidRoll triggers AdBanner at midRollInterval ticks

7. Ad System Architecture

The ad system spans all three services. The backend owns the configuration; the frontend fetches it and renders the appropriate ad component; the Go service is not involved in ad delivery.

Data Flow

```
appsettings.json
■■ 'Ads' section (Enabled, MidRollIntervalMinutes, PreRoll.*, MidRoll.*)
■
▼ IOptions injected into AdsController
GET /api/ads/config (ResponseCache: 300s, no auth required)
■
▼ fetchAdsConfig() in src/lib/api/ads.ts
■
▼ useAdConfig() hook (TanStack Query, staleTime: 5min)
enabled: subscriptionTier === 0 (Free tier only)
■
▼ VideoPlayer.tsx
■■ adConfig.preRoll → (before content, full-screen)
■■ adConfig.midRoll → (every midRollIntervalSeconds)
```

Backend: AdsController

File	apps/backend/src/StreamingPlatform.API/Controllers/AdsController.cs
Endpoint	GET /api/ads/config
Auth	None — anonymous access; ad config contains no secrets
Cache	[ResponseCache(Duration = 300)] — 5-minute HTTP cache header
Config binding	IOptions bound from appsettings.json 'Ads' section; registered via Configure() in Program.cs
Response DTOs	AdsConfigResponse, PreRollDto, MidRollDto — C# records

Frontend: AdPlayer Component

File	apps/frontend/src/components/player/AdPlayer.tsx
Trigger	Shown when showPreRoll === true and adsEnabled (subscriptionTier === 0 and config loaded)
Skip button	Locked for skipAfterSeconds (from config); shows countdown; unlocks to allow dismiss

Controls	Mute toggle, Learn More link (clickThroughUrl), ad progress bar, advertiser name
Completion	onComplete() fires on natural end or skip → VideoPlayer resumes main content
Conversion	'Upgrade to remove ads' link pointing to /account

Frontend: AdBanner Component

File	apps/frontend/src/components/player/AdBanner.tsx
Trigger	Fired when Math.floor(currentTime / midRollIntervalSeconds) changes to a new slot
Behaviour	Pauses main video; resumes on close. Auto-closes after durationSeconds (from config)
Close button	Locked for closeAfterSeconds (from config); shows countdown; unlocks to dismiss
Content	Image (imageUrl), headline, advertiser name, CTA button (Learn More), upgrade link

Updating Ad Campaigns (Zero-Downtime)

appsettings.Production.json

```
// Edit appsettings.json (or appsettings.Production.json):
"Ads": {
"Enabled": true,
"MidRollIntervalMinutes": 20, // was 15 — stretch interval
"PreRoll": {
"VideoUrl": "https://cdn.example.com/new-campaign.mp4",
"AdvertiserName": "Acme Corp",
"SkipAfterSeconds": 5
},
"MidRoll": {
"ImageUrl": "https://cdn.example.com/banner.jpg",
"Headline": "Summer Sale — 50% off Premium",
"DurationSeconds": 20
}
}
// Restart backend container. Frontend cache expires in 5 min.
// No frontend build or deployment needed.
```

8. Database Schema

Tables & Relationships

Table	PK	Notable Columns / Relations
users	uuid	email (unique), password_hash, name
profiles	uuid	user_id → users; name, avatar_url, pin_hash, is_kids_profile
subscriptions	uuid	user_id → users; tier (enum), starts_at, ends_at, stripe_subscription_id
genres	uuid	name, slug (unique)
content	uuid	title, slug, description, type (enum), required_tier, is_published, search_vector (generated tsvector)
content_genres	composite	content_id → content, genre_id → genres
seasons	uuid	content_id → content, season_number
episodes	uuid	season_id → seasons, episode_number, duration_minutes
video_assets	uuid	content_id / episode_id (nullable); quality (enum), hls_manifest_path, raw_file_path, status (enum), bitrate_kbps
subtitles	uuid	content_id / episode_id (nullable); language_code, file_url (WebVTT in MinIO)
watchlist	uuid	profile_id → profiles, content_id → content, added_at
watch_progress	uuid	profile_id, content_id, episode_id (nullable), position_seconds, is_completed (true at ≥90%), last_watched_at
channels	uuid	name, stream_url, category, is_live, thumbnail_url
live_events	uuid	channel_id → channels (nullable), title, scheduled_at, status (enum), stream_key

Key Indexes

```
-- Full-text search (generated tsvector column):
CREATE INDEX idx_content_search ON content USING GIN (search_vector);
CREATE INDEX idx_title_trgm ON content USING GIN (title gin_trgm_ops);

-- User activity sorted by recency:
CREATE INDEX idx_progress_profile ON watch_progress (profile_id, last_watched_at DESC);
```

```
CREATE INDEX idx_watchlist_profile ON watchlist (profile_id, added_at DESC);

-- Content browsing:
CREATE INDEX idx_content_type ON content (type, is_published);
CREATE INDEX idx_content_genre ON content_genres (genre_id);

-- Asset lookup by content or episode:
CREATE INDEX idx_asset_content ON video_assets (content_id, quality);
CREATE INDEX idx_asset_episode ON video_assets (episode_id, quality);
```

9. API Reference

Authentication

Method	Path	Auth	Description
POST	/api/auth/register	—	Register; returns { accessToken, refreshToken, user }
POST	/api/auth/login	—	Login; returns { accessToken, refreshToken, user }
POST	/api/auth/refresh	Refresh	Rotate refresh token; returns new token pair
POST	/api/auth/logout	Bearer	Invalidate refresh token in Redis

Content

Method	Path	Auth	Description
GET	/api/content	—	Paginated list; ?page&pageSize;&type;
GET	/api/content/featured	—	Featured titles (Redis cached 10 min)
GET	/api/content/trending	—	Trending (Redis cached 10 min)
GET	/api/content/genres	—	All genre slugs and names
GET	/api/content/genre/{slug}	—	Content by genre, paginated
GET	/api/content/{id}	—	Full detail: seasons, assets, subtitles
GET	/api/content/{id}/stream	Bearer	Returns master M3U8 URL + qualities + subtitles
POST	/api/content/{id}/upload	Admin	Returns presigned MinIO PUT URL for raw video

Ads

Method	Path	Auth	Description
GET	/api/ads/config	—	Returns full AdsConfigResponse (Enabled, PreRoll, MidRoll, MidRollIntervalSeconds). ResponseCache 300s.

Search

Method	Path	Auth	Description
GET	/api/search	—	?q=&type;=&genre;=&year;= pg_trgm similarity + tsvector full-text

Watch Progress

Method	Path	Auth	Description
GET	/api/progress	Bearer	All progress for active profile (Continue Watching)
GET	/api/progress/{contentId}	Bearer	Progress for one title
PUT	/api/progress/{contentId}	Bearer	Upsert position; marks completed at ≥90%

Watchlist

Method	Path	Auth	Description
GET	/api/watchlist	Bearer	All watchlist items for active profile
POST	/api/watchlist/{contentId}	Bearer	Add title; idempotent
DELETE	/api/watchlist/{contentId}	Bearer	Remove title

Users & Profiles

Method	Path	Auth	Description
GET	/api/users/me	Bearer	Current user info
PUT	/api/users/me	Bearer	Update name / email
GET	/api/users/me/profiles	Bearer	List all profiles
POST	/api/users/me/profiles	Bearer	Create profile (max 5)
PUT	/api/users/me/profiles/{id}	Bearer	Update profile
DELETE	/api/users/me/profiles/{id}	Bearer	Delete profile
GET	/api/users/me/subscription	Bearer	Active subscription details

Live & Channels

Method	Path	Auth	Description
GET	`/api/live`	—	Upcoming + live events list
GET	`/api/live/{id}`	Bearer	Live event detail + stream URL
GET	`/api/channels`	—	All TV channels
GET	`/api/channels/{id}`	Bearer	Channel detail + live HLS URL

Go Streaming Service (port 8080)

Method	Path	Auth	Description
GET	`/stream/{contentId}/master.m3u8`	?token=	Rewritten HLS variant playlist
GET	`/stream/{contentId}/{q}/prog_index.m3u8`	?token=	Quality-level playlist
GET	`/stream/{contentId}/{q}/{seg}.ts`	?token=	302 redirect to presigned MinIO URL
GET	`/live/{channelId}/stream.m3u8`	?token=	Live HLS stream proxy
POST	`/internal/transcode`	X-Key	Enqueue transcode job (backend → Go)
PATCH	`/api/admin/assets/{id}/status`	X-Key	Transcode complete callback (Go → backend)
GET	`/health`	—	Liveness probe

10. Infrastructure & DevOps

Docker Compose Services

Service	Image	Port(s)	Depends On
postgres	postgres:16-alpine	5432	—
redis	redis:7-alpine	6379	—
minio	minio/minio:latest	9000, 9001	—
minio-init	minio/mc:latest	—	minio
backend	apps/backend Dockerfile	5000→8080	postgres, redis, minio
streaming	apps/streaming Dockerfile	8080	minio, redis
frontend	apps/frontend Dockerfile	3000	backend, streaming
nginx	infra/nginx Dockerfile	80, 443	all

MinIO Buckets

Bucket	Access	Contents
videos-raw	Private	Original uploaded video files (any container format)
videos-hls	Public read	Transcoded HLS segments + master playlists
thumbnails	Public read	Poster and backdrop images (JPEG / WebP)
subtitles	Public read	WebVTT subtitle files per language per title

Nginx Routing

- location /api/ → proxy_pass http://backend:8080/ (REST API)
- location /stream/ → proxy_pass http://streaming:8080/ (HLS delivery)
- location /live/ → proxy_pass http://streaming:8080/ (live HLS)
- location / → proxy_pass http://frontend:3000/ (Next.js SSR + static)
- client_max_body_size 5G — permits large direct video uploads via Nginx
- proxy_read_timeout 3600s — prevents timeout waiting for long transcode callbacks

11. Security Architecture

Authentication & Authorisation

- Passwords hashed with bcrypt (cost factor 12). Never stored or logged in plaintext.
- Access tokens are short-lived (15 min) — limits blast radius of any token theft.
- Refresh tokens are opaque server-side strings in Redis; can be revoked instantly by deleting the Redis key.
- Refresh tokens are rotated on every use — each /auth/refresh invalidates the previous token.
- Subscription tier is embedded in JWT claims; enforced by both backend (policy attribute) and Go middleware.
- Ad config endpoint is deliberately anonymous — it contains no secrets and must load even for unauthenticated sessions.

Transport & Network Security

- All external traffic via HTTPS — Nginx terminates TLS; Let's Encrypt cert in production.
- Internal Docker network: services communicate over a private bridge network, not exposed externally.
- Backend ↔ Go service calls authenticated with X-Internal-Key header (shared secret, never in JWT).
- HLS segments gated by short-lived JWT query tokens; presigned MinIO URLs expire in 1 hour.

Input Validation & Injection Prevention

- All inputs validated by FluentValidation pipeline behaviour before reaching MediatR handlers.
- EF Core uses parameterised queries throughout — no raw string SQL concatenation.
- Raw SQL in SearchController uses Npgsql parameters (@q) — immune to SQL injection.
- Frontend form inputs validated with zod schemas before API submission.
- File uploads restricted to video MIME types; size validated server-side.

Phase 2 Security Enhancements

- Widevine (Chrome/Android) + FairPlay (Safari/iOS) DRM for Premium tier content.
- Rate limiting on auth endpoints — Redis sliding window, 5 attempts per 15 minutes.
- OWASP Content Security Policy headers added via Nginx.
- Automated dependency audits: Dependabot for NuGet, npm, Go modules; SAST in CI.

12. Scalability & Performance

Horizontal Scaling Strategy

Next.js Frontend	Stateless — scale replicas behind Nginx. Server-rendered browse pages cached at CDN edge with stale-while-revalidate. Ad config cached 5 min at browser.
.NET Core Backend	Stateless — session data in Redis, no in-memory state. Npgsql connection pooling (10–100). Featured/trending content cached in Redis (10 min TTL). Ad config cached 5 min via ResponseCache.
Go Streaming	Worker pool controls FFmpeg concurrency (default: 3 simultaneous processes). HLS serving = presigned redirects — no byte proxying. Scale replicas freely; MinIO is shared state.
PostgreSQL	Read replicas for browse/search traffic. Write traffic to primary only. GIN indexes keep pg_trgm search fast at 1M+ rows.
Redis	Redis Cluster for high availability. Separate logical DBs for tokens (TTL-critical) vs. content cache vs. rate limiting.
MinIO / CDN	MinIO in production fronted by CloudFront. HLS .ts segments (~200KB each) are highly cacheable with long TTLs. Ad creative assets (video, images) served from CDN.

Performance Targets

Metric	Target
API response time (p95)	< 100ms
Search response time	< 200ms (PostgreSQL GIN)
HLS manifest response	< 50ms (Redis-cached path resolution)
Ad config response	< 30ms (IOptions, in-memory binding)
Transcode time (90-min movie)	< 25 min (3 quality levels, parallel FFmpeg)
Player startup time	< 3 seconds on 5 Mbps connection
HLS segment cache hit rate	> 85% (CloudFront / Nginx proxy cache)
Uptime SLA	99.9% monthly

13. Development Workflow

Local Setup

```
git clone https://github.com/remon024/streming && cd streming
cp .env.example .env # fill in secrets
make dev # starts all 7 Docker containers
make migrate # runs EF Core migrations
make seed # loads sample genres + content

# Access points:
http://localhost:3000 # Next.js frontend
http://localhost:5000/swagger # Backend Swagger UI
http://localhost:9001 # MinIO console
http://localhost:8080/health # Go service health check
```

Ad System Verification Checklist

```
1. GET http://localhost:5000/api/ads/config
→ Returns JSON with enabled:true, midRollIntervalSeconds:900, preRoll, midRoll

2. Log in as a Free tier user (subscriptionTier: 0)
3. Navigate to /watch/{contentId}
→ AdPlayer (pre-roll) appears before video starts
→ Skip button is disabled for first 5 seconds, shows countdown
→ Skip button enables; clicking it dismisses ad and starts content

4. Fast-forward to 15:01 (or change MidRollIntervalMinutes: 1 in config to test quickly)
→ Video pauses; AdBanner appears at bottom of player
→ Close button locked for 5 seconds, then dismissible
→ Video resumes after close

5. Log in as a paid user (subscriptionTier: 1+)
→ GET /api/ads/config is never called (useAdConfig disabled)
→ No AdPlayer, no AdBanner — zero ad components rendered

6. Update VideoUrl in appsettings.json, restart backend
→ Wait 5 minutes (or clear browser TanStack Query cache)
→ New ad creative plays on next /watch page load
```

End-to-End Content Upload & Stream Test

```
1. POST /api/auth/register → get access token
```

```
2. POST /api/content/{id}/upload → get presigned MinIO PUT URL
3. PUT → upload raw video file
4. Backend calls POST http://streaming:8080/internal/transcode
5. Go service transcodes → uploads HLS → PATCH /api/admin/assets/{id}/status
6. Poll GET /api/content/{id} until videoAssets[].status === 'ready'
7. GET /api/content/{id}/stream → returns masterM3u8Url
8. Open /watch/{id} → VideoPlayer loads master.m3u8
9. Confirm 360p / 720p / 1080p levels in QualitySelector
10. Play 30s → verify PUT /api/progress/{id} fires via browser Network tab
11. Refresh page → player resumes at correct position
```

— End of Document —